

Tutorial on python noisify

Introduction

The code to noisify input image can be found in main.py:

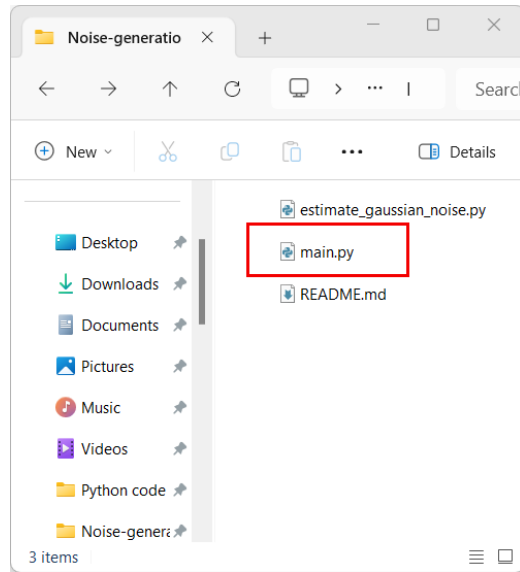
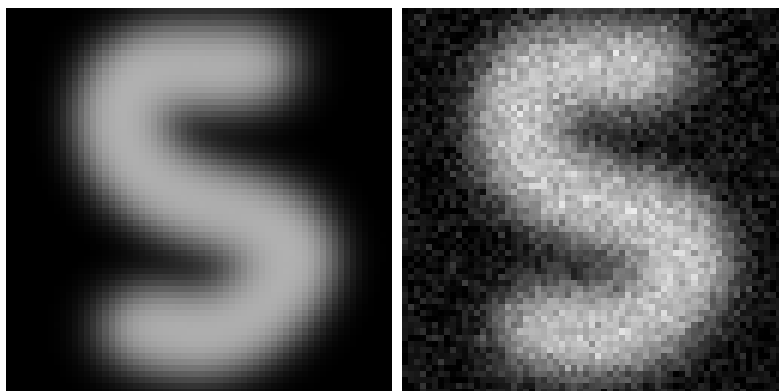


Figure 1

The code allows for a simulation of a measurement by introducing shot noise (Poisson distribution) and thermal noise (Gaussian distribution) into a grayscale image. The output is an image file simulating what a single photon sensitive camera would produce.

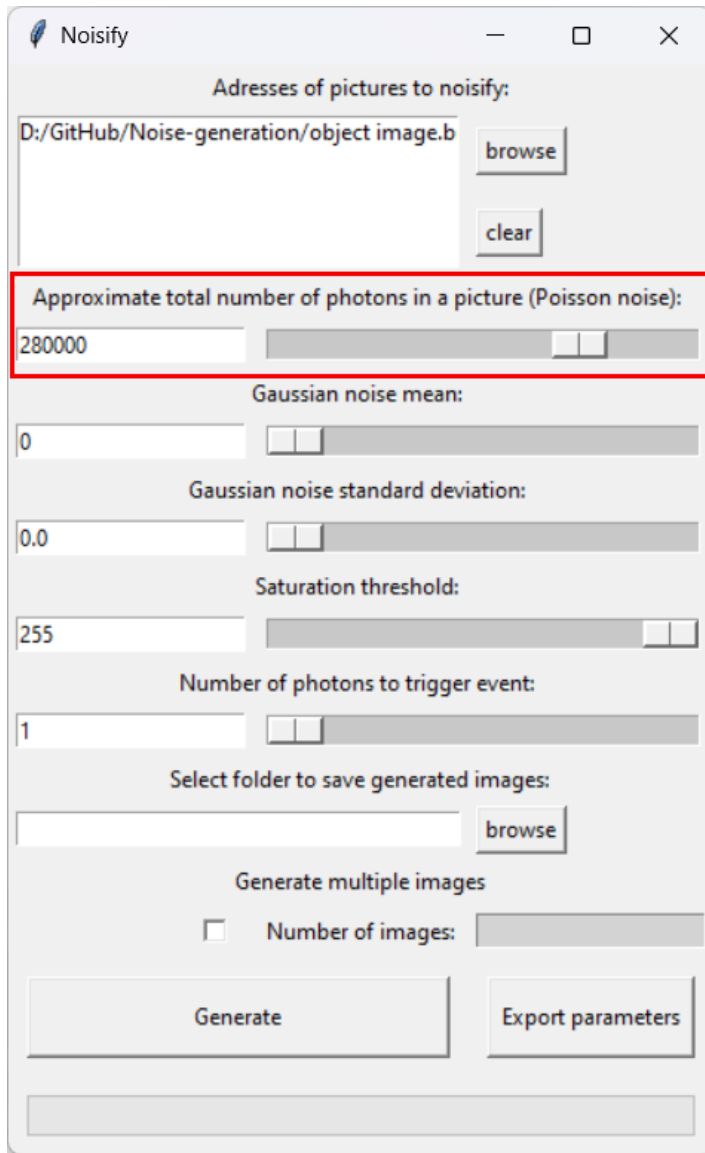
The grayscale image can take values from 0 to 255. We consider the grayscale value of a single pixel its number of events. By default, 1 event corresponds to 1 photon count. This can be changed later (see Section 2).



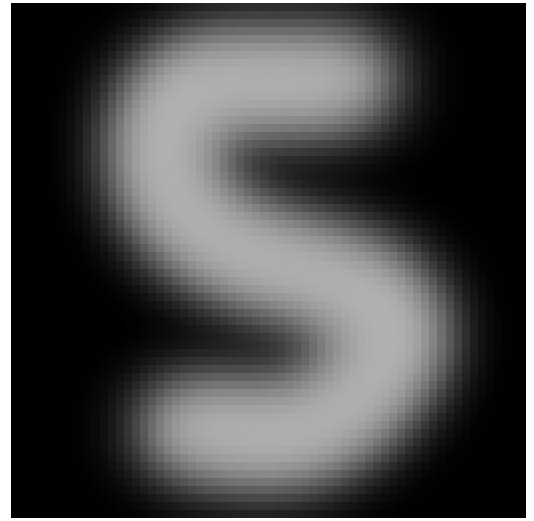
(a) Object (ground truth) (b) Example of noisified image

Figure 2

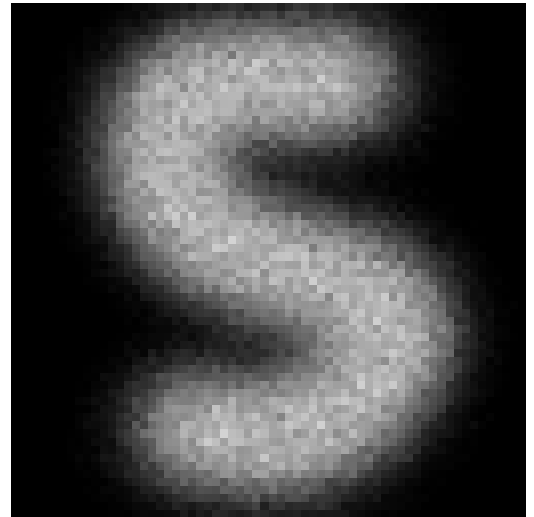
1 Shot noise



(a) Poisson noise parameter



(b) Object (ground truth)

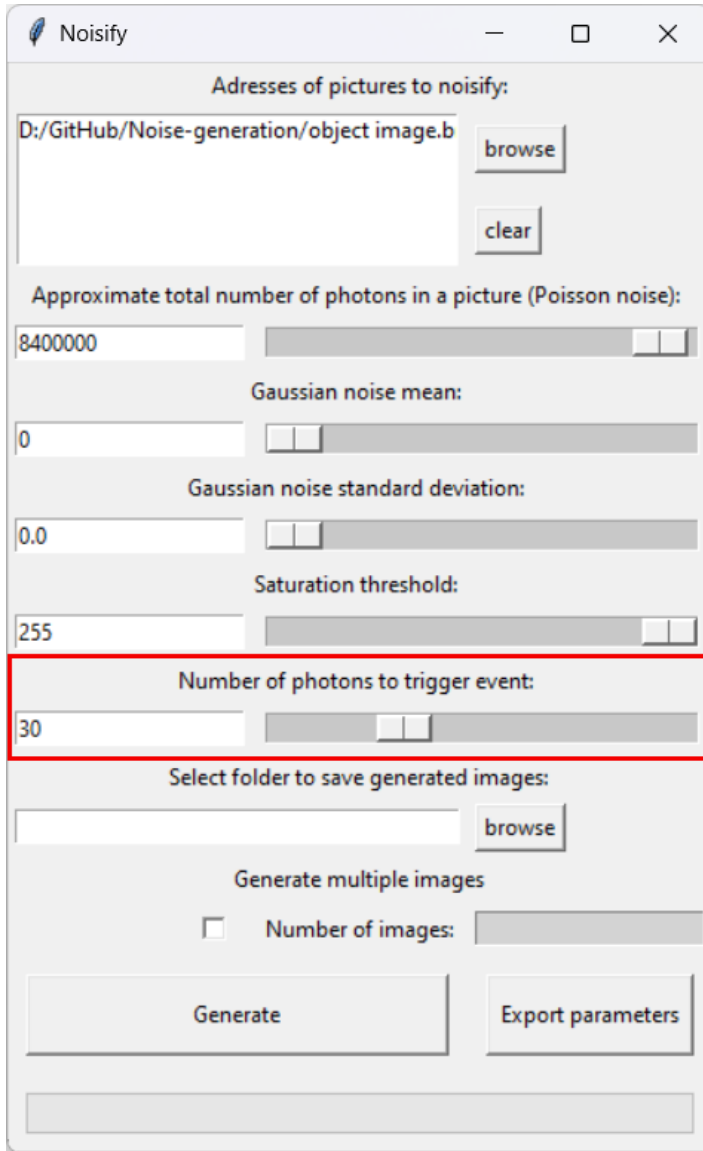


(c) Image with shot noise (~ 280.000 total photons)

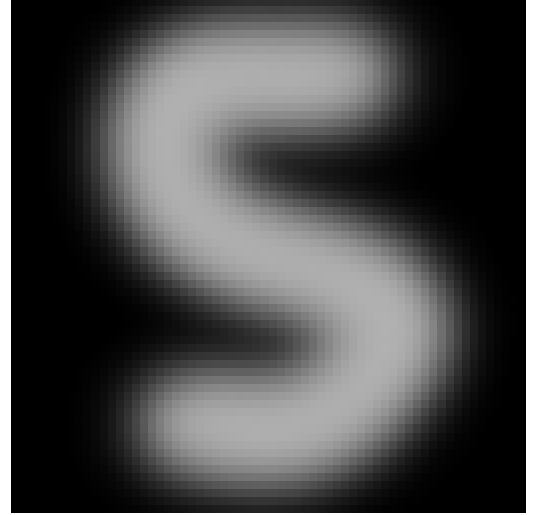
Figure 3: Effect of Poisson noise

Given a limited number of photons, the number of events recorded by a single photon camera is probabilistic for each pixel. We model this with a Poisson distribution, wherein the lambda parameter is given by the grayscale value of the input image at that particular pixel.

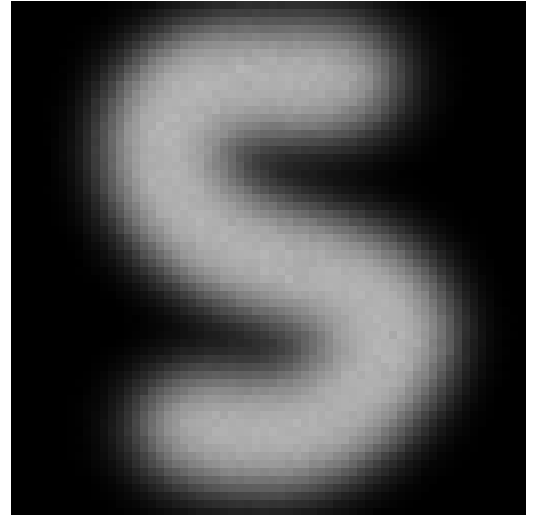
2 Number of photons to trigger event



(a) Number of photons to trigger event



(b) Object (ground truth)



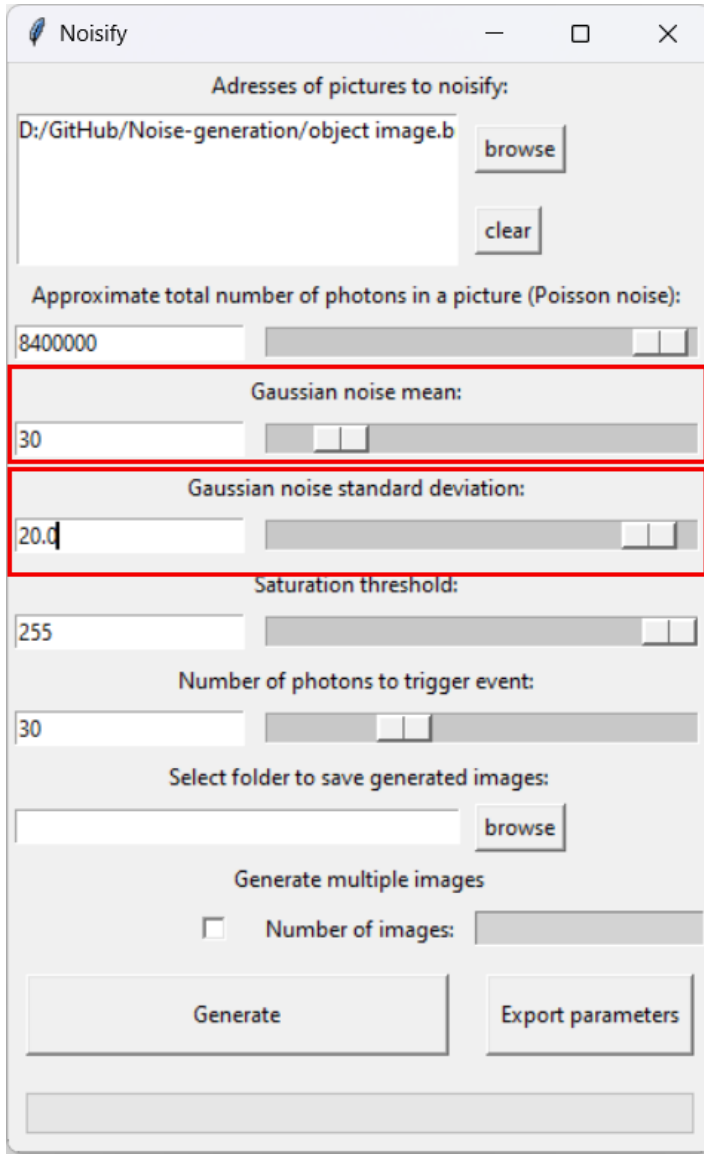
(c) Image with shot noise ($\sim 8.400.000$ total photons)

Figure 4: Same simulation as in fig.3, with 30 times more photons, and 30 photons needed to trigger an event

The number of photons to trigger event parameter allows to simulate a higher number of photons without hitting the saturation threshold. For instance, with a "number of photons to trigger event" set to 30, a grayscale value of 7 on a certain pixel means that $30 \times 7 = 210$ photons hit the detector.

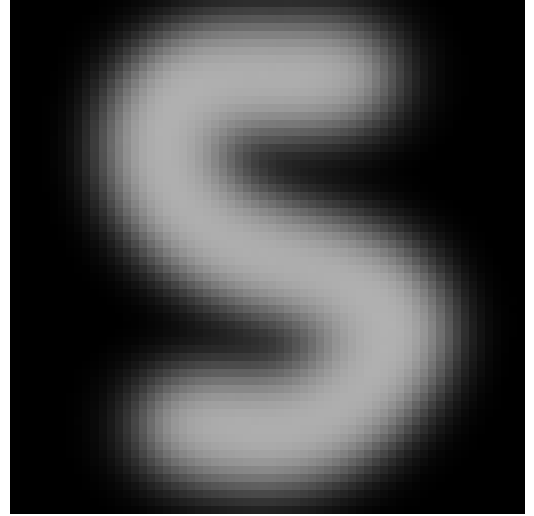
The more photons are used during a single measurement, the more the distribution of intensity approaches the ground truth (assuming only shot noise, with no thermal noise), as illustrated by fig.4.

3 Thermal noise

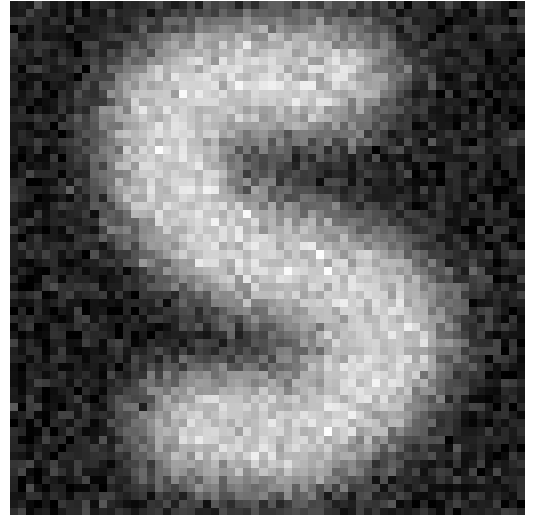


The screenshot shows a window titled "Noisify" with several input fields and sliders. The "Gaussian noise mean" and "Gaussian noise standard deviation" fields are highlighted with a red rectangle. The "Gaussian noise mean" field contains the value 30, and the "Gaussian noise standard deviation" field contains the value 20.0. Other parameters include "Approximate total number of photons in a picture (Poisson noise)" set to 8400000, "Saturation threshold" set to 255, and "Number of photons to trigger event" set to 30. There are buttons for "browse", "clear", "Generate", and "Export parameters".

(a) Gaussian noise parameter



(b) Object (ground truth)

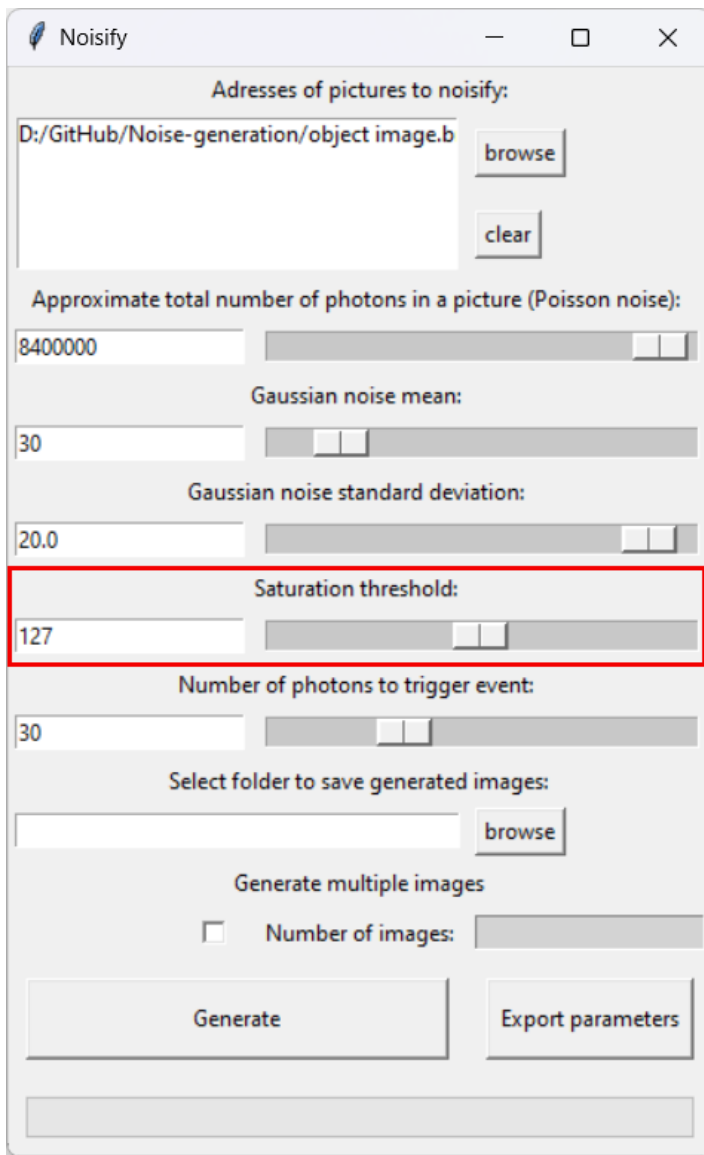


(c) Image with Gaussian noise

Figure 5

On top of the events due to shot noise, we generate Gaussian noise meant to simulate random counts of the detector itself (mostly due to thermal effects). This is modeled with i.i.d. Gaussian distributions for each pixel, with a mean value (interpreted as a constant background), and a standard deviation. If the number of events happens to be negative, it gets set to 0.

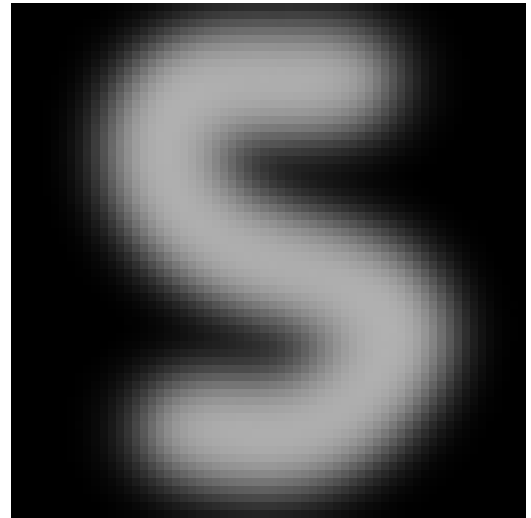
4 Saturation threshold



The screenshot shows the 'Noisify' application window with the following parameters:

- Adresses of pictures to noisify: D:/GitHub/Noise-generation/object image.b
- Approximate total number of photons in a picture (Poisson noise): 8400000
- Gaussian noise mean: 30
- Gaussian noise standard deviation: 20.0
- Saturation threshold: 127** (highlighted with a red box)
- Number of photons to trigger event: 30
- Select folder to save generated images: (empty)
- Generate multiple images: ☐ Number of images: (empty)
- Buttons: Generate, Export parameters

(a) Saturation threshold parameter



(b) Object (ground truth)



(c) Saturated simulated image (max value is 127)

Figure 6

The maximum number of events for all pixels can be set using the saturation threshold. With standard images, it can at most be 255.

5 Estimate Gaussian noise

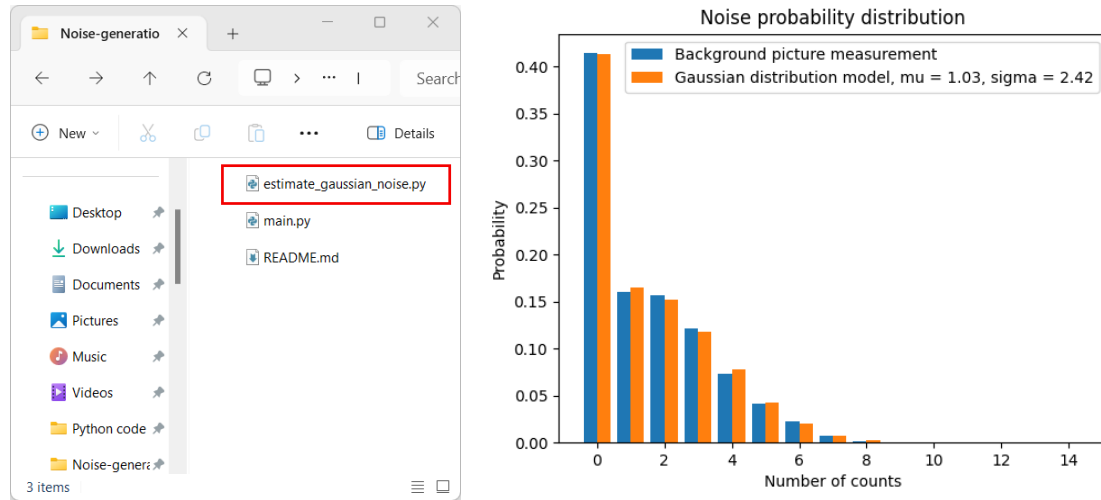


Figure 7

To calculate mu and sigma parameters from data, you can run "estimate_gaussian_noise.py" and provide a background image that contains only dark counts. The program will automatically calculate the distribution and estimate the thermal noise parameters.